DRISHTI · दृष्टि · SIGHT

Drishti VaR engine
Architecture, data contract and how VaR works

Any set of trades or positions, in any reporting currency

Ashutosh Sinha

October 2026

Release 2.9.4 · historical simulation over 509 scenario days, window and confidence configurable

IN THIS DECK

1 How VaR works in Drishti
2 Architecture and data flow
3 Any input: PVs, P&L, intraday, contracts
4 The VaR UI and capabilities
5 Scale, Delta Lake, scale-out
6 Governance and roadmap

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

Executive summary

112.04m
VaR 99%, 1 day, USD: the demo bank on 2026-09-30 (the 6th worst of 509)

160.14m
ES 97.5%, USD: the mean of the 13 worst scenario days

509
historical scenario days (2024-09-17 to 2026-09-29), equal weights

9,572
live trades in 7,984 derived positions, 88 nodes, 28 portfolios

< 1 ms
a node's VaR from its float64 aggregate; 572 ms to load the date

What is it?	A VaR engine in the Drishti server: historical simulation over 509 scenario days (or a set window), VaR 99% (95% by setting) and ES 97.5%, 1 day, on demand for any set of trades or positions: the bank, a legal entity, desk, book, a portfolio, a rule or a basket.
Every input	**One engine, every input: scenario PVs or path P&L, by trade or by position, any currency, files or Kafka. Source systems keep their own formats.**
How does it work?	It never reprices. The pricing systems send scenario PVs (in each trade's or position's own currency) or path P&L, at trade or position level, through files or Kafka. Drishti turns each into one internal form, a P&L vector per trade or position, converting PVs with scenario FX into any reporting currency. A scope's vector is the sum of its trades' or positions' vectors, and VaR, ES and every contribution are read from that one vector.
Can I trust it?	Every answer names its inputs' versions, batch ids, scenario set, method and memory tier; components add up to the VaR exactly; every node gets an official end-of-day record, append-only, kept 7 years.
Does it scale?	Measured: 1 million rows summed in 58 ms (heap tier); 100,000 × 5 dates on the tiered store, a node in 0.48 ms. Sized for 1.3M positions and 100,000 portfolios on slides 12-13; sharding is designed, not built.

How VaR works in Drishti: add vectors, then read the tail

1 A P&L vector per trade
509 numbers: its P&L if the market moved as on each scenario day

→

2 Add the scope's vectors
any scope is a set of trades; overlaps count once, longs and shorts net

→

3 Sort the 509 P&Ls
worst first; equal P&Ls: the older scenario first

→

4 Read the tail
VaR: minus the 6th worst; ES: minus the mean of the 13 worst

Worked example

ten scenarios, α = 80% so k = 2; one book holding P1 (T1 long, T2 short) and P2 (T3)

Vector	0	1	2	3	4	5	6	7	8	9
T1	5	−8	3	−12	6	−2	4	−5	7	1
T2	−2	3	−1	4	−3	1	−2	2	−3	0
P2 = T3	−4	2	−6	1	−9	3	−1	0	2	−7
P1 = T1 + T2	3	−5	2	−8	3	−1	2	−3	4	1
Book = P1 + P2	−1	−3	−4	**−7**	**−6**	2	1	−3	6	**−6**

VaR
6
2nd worst of the book: −6

ES
6.5
mean of −7 and −6

Component VaR
1.26 + 4.74
P1 + P2 = 6: Euler, adds up

Diversification
6
standalone 5 + 7 − 6

Incremental, P1
−1
a hedge: removing it raises VaR

At N = 509 the same steps use the 6th worst for VaR, the 13 worst for ES and ranks 4 to 8 for components. Defaults shown: window and confidence are settings (next slide).

The measures at N = 509

Measure	Definition
VaR 99%	minus the 6th worst P&L
ES 97.5%	minus the mean of the 13 worst
Component VaR	Euler on ranks 4–8; adds up to VaR
Component ES	mean over the 13 worst; adds up to ES
Incremental	VaR(scope) − VaR(scope − part)
Marginal	component ÷ |size| × 1,000,000
Diversification	Σ standalone VaR − VaR
Limits	per node, portfolio; amber 75%, red 90%
Backtest	250 days, Basel light, Kupiec, Christoffersen

CONFIGURABLE · 2.9.4

The window, the confidence, the currencies and the names are settings

Scenario window

`drishti.var.scenario-window: all | N`

The newest N scenarios of the set (all: 509). Per request, window=250 shows a 1-year view beside the 2-year one: labelled on every answer, never official.

Window	VaR 99%	ES 97.5%
all (509)	6th worst	mean of 13
250	3rd worst	mean of 7

Confidence

`DRISHTI_VAR_CONFIDENCE · DRISHTI_VAR_ES_CONFIDENCE`

VaR 99% by default, 95% by configuration; ES 97.5% by default; both also per request. Every label follows the answer (VaR 95% · 1 day). Backtesting stays at 99%.

VaR at	509 days	250 days
99% (default)	6th worst	3rd worst
95%	26th worst	13th worst

Choosing the window banks typically use 1 year (≈ 250 days, the regulatory minimum) or 2 years (≈ 500). A shorter window reacts faster to new volatility; a longer one is steadier and keeps more stress days. Compare both with window= before changing the setting.

Currencies, built ahead

`drishti.var.currencies`
USD, EUR, CAD, JPY, GBP, built in the background after the USD book

`currency-retention-dates`
5: the newest dates kept; older ones built again on first ask

Names

`DRISHTI_VAR_ROOT_NAME`
the top of every VaR tree (root-names: per hierarchy)

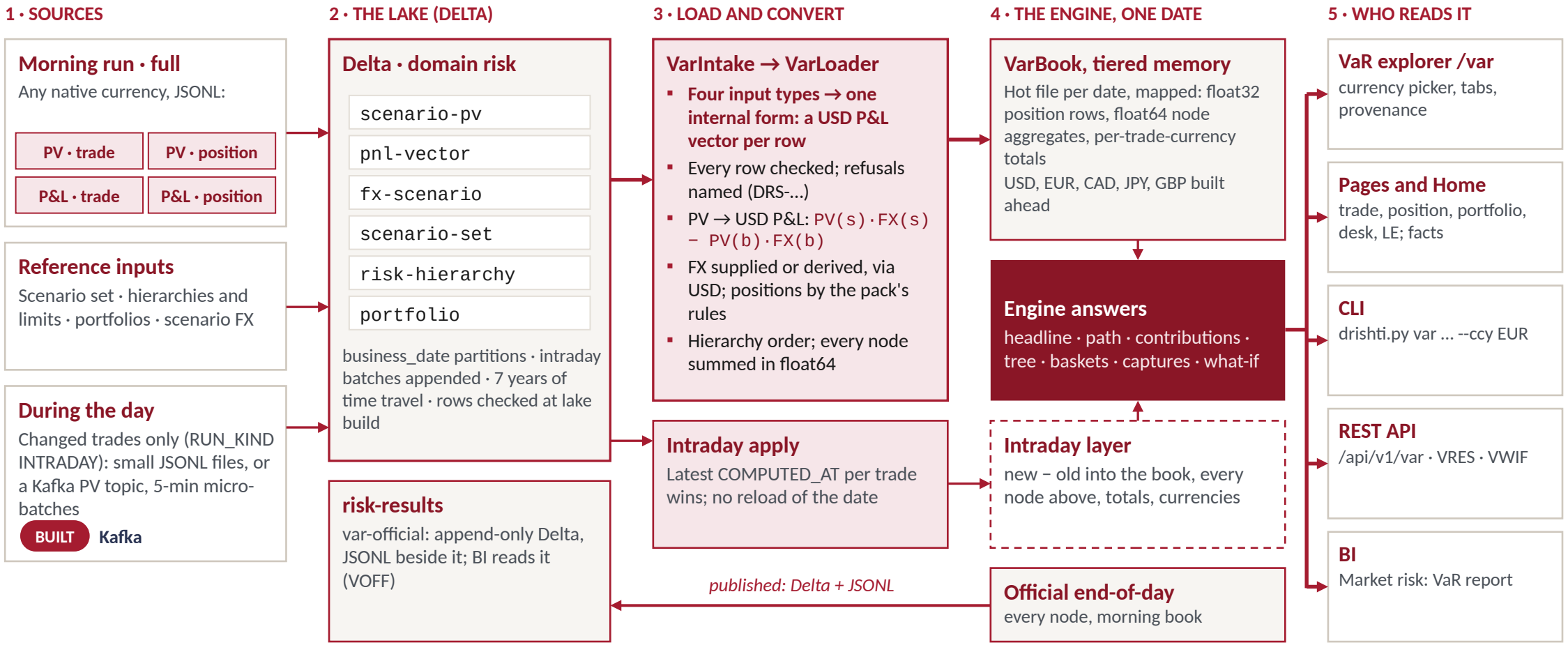
`config/demo/names.yaml`
the demo's bank and legal-entity names

Inputs and intraday

`drishti.var.inputs`
optional pin per source system: form and grain

`intraday.kafka.window`
5m; early flush at max-trades (500)

From the pricing systems to every screen, in one direction



Each batch's data-load signal `POST /api/v1/packs/market-risk/loads` *tells the engine: a full run reloads the date; an intraday batch is applied on top. The lake is the record: a restart rebuilds everything, the intraday layer included.*

ANY INPUT CONTRACT

One engine, every input: source systems keep their own formats

	Scenario PVs native currency	Path P&L vectors any currency
By trade	✓ BUILT 2.9.4	✓ BUILT since 2.8.0
By position	✓ BUILT 2.9.4	✓ BUILT 2.9.4

Delivery routes

Files, JSONL — BUILT
the morning run; small intraday batches appended

Kafka, micro-batched — BUILT
JSONL messages; 5-min window, early flush, Apply now

Intraday, either route — BUILT
changed trades only, as new − old; latest wins

```
{"VALUE_TYPE":"PV",  "GRAIN":"TRADE",    "TRADE_ID":"1000279", "CURRENCY":"EUR", "BASE_PV":…, "PVS":[…509…], "COMPUTED_AT":"…", …}
{"VALUE_TYPE":"PV",  "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"JPY", "BOOK":"…", "BASE_PV":…, "PVS":[…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"TRADE",    "TRADE_ID":"MX-…",    "CURRENCY":"USD", "PNL":[…509…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"USD", "PNL":[…], …}
```

Mix and switch

- Each system sends its own form and grain; systems mix on one date.
- A system may switch day to day; each date loads as it stands.
- Old files: `drishti.py convert --set …`

```
# optional cross-check
drishti.var.inputs:
  ENDUR:   {form: pv,  grain: trade}
  CALYPSO: {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

Guarantees

- One form and grain per system per date (DRS-5210): no double counting
- Fields missing or wrong (DRS-5208), or against the pin (DRS-5209): refused, never guessed
- Provenance shows what each system sent

Caveat a P&L vector sent without its PV converts to another reporting currency only approximately (at each scenario's rate, without the FX move of its base value; counted as `tradesWithoutPv`). Scenario PVs give exact multi-currency VaR.

From scenario PVs in each trade's currency to VaR in any currency

Inputs

- Pricing systems send each trade's 509 scenario PVs and base PV in the trade's own currency (`scenario-pv`). The USD P&L-vector form stays supported; a date may mix both, a trade in one form.
- Scenario FX: the batch's own (`fx-scenario`) when sent, else derived from the FX history. Every cross goes via USD. Horizon: 1 day.
- Converted when the date loads; positions, nodes, captures and official records do not know which form a trade came in.

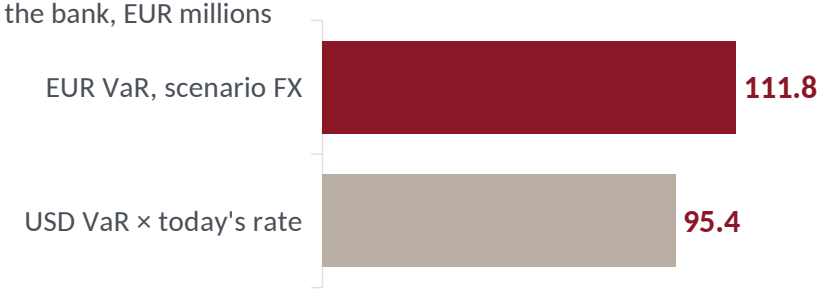
```
USD P&L      PV(s)·FX_c(s) − PV(base)·FX_c(base)
FX derived   FX(s) = FX(base) · H(d_s) / H(d_s − 1)
In r         (p + P&L_USD(s)) / FX_r(s) − p / FX_r(base)
Node in X    Σ_c [PVsum_c(s)·FX_c→X(s) − PVbase_c·FX_c→X(base)]
```

Reporting currencies

- Built ahead in the background: USD · EUR · CAD · JPY · GBP (`drishti.var.currencies`); the newest 5 dates kept; any other currency with FX on first ask.
- Every node keeps its PVs summed per trade currency: a headline in any currency is exact to the cent.
- Trade pages say the Trade currency (EUR · as priced); official records in USD, others per setting.

Why EUR VaR ≠ USD VaR × today's rate

the bank, EUR millions



The bank on 2026-09-30 holds a net 3.65 bn USD of value; on the scenario days it moves against the euro, a risk a euro-based owner carries and a dollar-based one does not.

Ccy	Bank VaR 99%	Bank ES 97.5%
USD	112,041,646	160,143,809
EUR	111,837,045	137,631,546
CAD	189,801,404	230,680,413
JPY	20,883,226,318	23,578,418,999
GBP	104,139,554	125,595,958

On the demo: the 1,339 Endur trades come as scenario PVs (1,089 USD, 246 EUR, 4 CAD). Bank VaR 112,035,906 → 112,041,646 and ES 160,132,107 → 160,143,809: the FX move of the 250 EUR and CAD trades. A USD trade reproduces its vector exactly.

Intraday: only the changed trades, applied as new – old up the tree

Morning full run
RUN_KIND FULL: every live trade; the date loads, hot file built; official records taken from it

→

Changed trades
Amended, new, cancelled; same scenario set; own BATCH_ID, later COMPUTED_AT

BUILT JSONL files

→

Lake: appended batch
Files: drishti.py var intraday --load (lake build --mode append). Kafka: one batch per window

BUILT Kafka micro-batches

→

Apply: new – old
Into the book, every node above (MGMT and LEGAL), the currency totals and built currencies

→

Answers say intraday
Badge in the status line, intraday on every answer; morning hot file never changed

Case	What the engine does
Same trade in several batches	latest COMPUTED_AT wins across files and Kafka; an equal one keeps the first
COMPUTED_AT before the morning run	ignored and counted
Cancel (CANCELLED, vector null)	trade removed from every scope
New trade	book from TT_BOOK, position by the position rules
Moved to another book or position	old ordinal cancelled, added in its new place
Another scenario set	row refused (DRS-5203); the date stays as it is
A full run of the date	date reloads; the intraday layer is dropped

Cost of a batch ∝ trades changed × nodes above × currencies built, never the size of the book.

Kafka, as built in 2.9.4

- A message is JSONL text: one or more records, PVs or P&L, a cancel as a marked line
- A window every 5 min (drishti.var.intraday.kafka.window); early flush at 500 trades; admin “Apply now”
- Each window written to the lake as ONE intraday batch, then applied like a file
- Replaying the topic after a restart writes nothing twice

The guarantee incremental = the same data loaded as one run, to the cent, on every node

Measured: 200 batches of 3 trades on one date; a restart equal too; batch time does not grow. Official numbers come only from full runs.

Six input contracts, JSONL on the wire, every check named

JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; each line exactly its contract's fields. Key fields of the as-built contracts:

scenario-pv SPV · a trade × date × set	
TRADE_ID … TT_BOOK	the trade; its LE, desk, book
CURRENCY · BASE_PV	PVs' currency; value today
SCENARIO_PV [509]	per scenario; null = failed
RUN_KIND	FULL or INTRADAY
SCENARIO_SET_ID	and BATCH_ID: the run

pnl-vector PNLV · a trade × date × set	
VECTOR_ID · TRADE_ID	PV-<trade>-HS-509
PNL [509]	USD P&L, gain positive
PV · REPORTING_CCY	USD value today; USD
STATUS	ok, proxied or failed
RUN_KIND	as scenario-pv

fx-scenario FXS · a currency × date	
FX_SCENARIO_ID	FXS-<CCY>-HS-509
CURRENCY	AUD, CAD, EUR, JPY …
RATE	today, USD per one unit
RATES [509]	per scenario, the set's order
not sent?	derived from the FX history

scenario-set SSET · one row a date	
SCENARIO_SET_ID	HS-509
SCENARIO_DATES	509 days, oldest first
WINDOW_START … END	2024-09-17 … 2026-09-29
HORIZON_DAYS	1
REVALUATION	full or sensitivities

risk-hierarchy HIER · a node × date	
NODE_ID	MGMT~<id> or LEGAL~<id>
HIERARCHY · LEVEL	MGMT/LEGAL; bank…book
PARENT_ID · NAME	the tree; display name
BOOKS	every book beneath
LIMIT_VAR	1-day 99% limit, USD

portfolio PORT · a portfolio × date	
PORTFOLIO_ID · NAME	PF-…; scope PORT:<id>
TYPE	HIERARCHY, RULE or BASKET
DEFINITION	NODE, RULE or MEMBERS
OWNER	a user or a team
LIMIT_VAR	the portfolio's limit, USD

BUILT **Required on every record (2.9.4):** VALUE_TYPE, GRAIN with its id, CURRENCY, SOURCE_SYSTEM; GRAIN matches the id. Old files: `convert --set`

DRS-5201 length, null, NaN	**DRS-5206** a PV failed	**DRS-5207** no scenario FX	**DRS-5208** record fields wrong	**DRS-5209** against the pin	**DRS-5210** 2nd form or grain	**DRS-5202** trade twice	**DRS-5203** two runs

One explorer for any scope, and the same answers everywhere else



VAR DESK DESK-RATES IN EUR: the scope tree and tiles in EUR, the basket tray, the P&L path, and How this number was built naming the reporting currency, the input form, the scenario FX and the crosses via USD (2026-09-30).

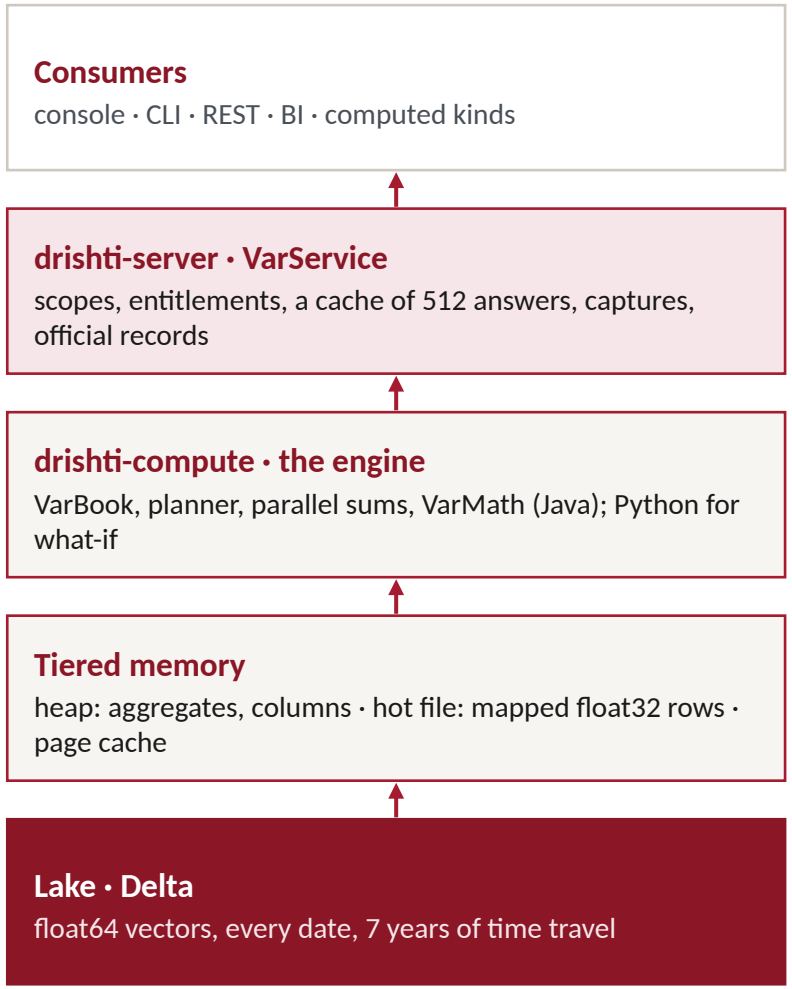
9–14 ms click to painted, a desk, answers fetched ahead (target 300 ms)

390 px on a phone the three columns stack, no sideways scroll

- **Scope tree and six tiles** MGMT or LEGAL tree with VaR, ES and limit bars; VaR, ES, sum of standalone, diversification, limit use, backtest light.
- **Five tabs** P&L path (509 days, worst 25, who lost on a day) · VaR/ES history and backtest · contributions · captured results · In other currencies.
- **Reporting currency picker** USD, EUR, CAD, JPY, GBP and More…; every panel follows; kept in the link; VAR … IN EUR.
- **How this number was built** the scope as resolved, trades, hidden and unknown ids, batch, data versions, method, tier, timing.
- **Baskets and pre-deal what-if** any mix of portfolios, trades, counterparties and rules, live; a booked trade's vector × a factor through the Python engine.
- **Captures, official records** immutable captures, compare with now; each node's official history.
- **Everywhere else** trade (Trade currency), position, portfolio, desk and LE pages; drishti.py var … --ccy; /api/v1/var; VRES and VWIF; the BI VaR report; Home facts.
- **Governed** root name drishti.var.root-name; per-node positions or aggregate access; masks; every answer audited.

HOW IT SCALES

Why it is fast: vectors add, one date stays hot, sums run on every core



Vector math
509 doubles = 4,072 B a trade; hot rows float32 (2,036 B); every sum in float64

Memory-mapped hot file
one file a date; only touched pages resident; 2 dates open, 60 kept on disk

Planner over aggregates
a node is its aggregate; Markets minus three trades = 4 vector operations

Parallel sums
slices on every core, added in a fixed order: the same bits every time

Currencies built ahead
five books in the background; 21.4 MB a currency a date on the demo

Per-trade-currency totals
a headline in any currency from a node's few currency sums, exact

Intraday increments
cost ∝ trades changed; Kafka: one commit and one apply per window

Cluster and history
Drishti*net replicas; VaR sharding designed; Delta history

Measured	Time	On
A node's VaR and ES	0.48 ms	tiered store, 100,000 trades
The whole bank from float32 rows	3.4 ms (≈ 60 GB/s)	tiered store, 100,000 trades
The whole bank from float64 rows	58 ms (≈ 70 GB/s)	heap tier, 1,000,000 trades
A basket of 50,000 scattered positions	2.6 ms + ≈ 45 ms ids	tiered store, 100,000 trades
First load of a date (after the lake read)	0.8–1.2 s	tiered store, 100,000 trades
Loading the demo date	572 ms	9,572 trades, the server

Bank scale, from first principles and the measured figures

Per business date	Arithmetic	Result	Basis
Vectors, float64 (the lake)	1.3M × 509 × 8 B	5.29 GB	arithmetic
Vectors, float32 (hot rows)	1.3M × 509 × 4 B	2.65 GB	arithmetic
Hot file, USD book	237.7 MB per 100k × 13	≈ 3.1 GB	*estimated*
Heap, one open date	≈ 80 MB per 100k × 13	≈ 1.0 GB	*estimated*
First load, after the lake read	0.8–1.2 s per 100k × 13	≈ 10–16 s	*estimated*
Reopen a date from its file	23 ms per 100k × 13	≈ 0.3 s	*estimated*
Whole bank from rows, resident	2.65 GB at ≈ 60 GB/s	≈ 45 ms	*estimated*
... cold, from NVMe	2.65 GB at ≈ 3 GB/s	≈ 0.9 s	*estimated*
A node, any size	its float64 aggregate	≈ 0.5 ms	**measured**
4 extra currencies, disk	250 MB × 4 per 100k × 13	≈ 13 GB a date	*estimated*
... 5 retained dates; build	13 GB × 5; 4.3 s per 100k × 13	≈ 65 GB; ≈ 56 s	*estimated*
USD hot files, 60 dates	3.1 GB × 60	≈ 185 GB disk	*estimated*
Lake, a year (Parquet ≈ 0.75×)	5.29 GB × 0.75 × 252	≈ 1 TB; ≈ 7 TB in 7 y	*estimated*

Position grain (2.9.4) keeps rows = positions: no trade matrix in the hot file. *Estimated = scaled linearly from the measured 100,000-trade figures (VAR_GUIDE 27.3, 15.11); not measured at 1.3M.*

100,000 portfolios

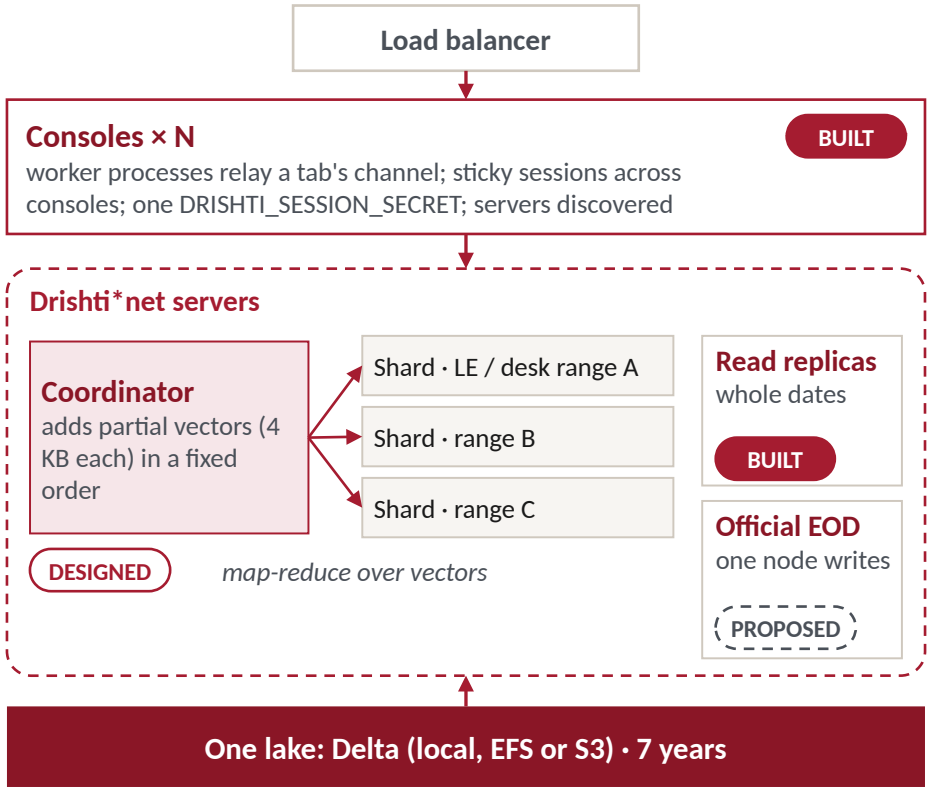
- A portfolio is the sum of its positions: the cost is all portfolios' positions together × 509 additions.
- At 130 positions each (assumed): 13M rows, 26.5 GB of float32 → ≈ 0.45 s at 60 GB/s, ≈ 5 s scattered; per currency, estimated.
- Roll-ups: a HIERARCHY portfolio is a node aggregate; whole books cost one add each; a node minus a few rows is aggregate − rows (built).
- Store them: 100k × 509 × 8 B = 407 MB a date a currency; float32 halves it (proposed).

Totals and intraday

- Currency totals: 510 × 8 B per node per trade currency; 5,000 nodes × 10 ≈ 204 MB (assumed shape).
- Per changed trade: ≈ 9 node vectors × 5 currency books ≈ 45 additions of 509: microseconds; the lake append dominates (≈ 1 s a batch).
- A headline in any currency: a node's few trade-currency sums, converted: microseconds.

Delta for history and reproducibility, Drishti*net for scale-out

Partition by business_date; 7 years of time travel full run replaces a date; restatement = new commit	BUILT
Vector column list<double>, float64 standard Parquet; a sum reads one column	BUILT
Compaction + checkpoints (lake maintain) many small intraday commits stay fast	BUILT
Sort by LE, desk, book; ZSTD + byte-stream split a node is a contiguous run of rows	DESIGNED
Z-order or liquid clustering by book and portfolio skip files for a desk's or portfolio's read	PROPOSED
Published node and portfolio vector table BI and peers read sums, not 1.3M rows	PROPOSED
Change data feed on the input tables follow intraday deltas downstream	PROPOSED
Delta inputs on S3 (reading) the Delta plugin reads s3:// tables	BUILT
Iceberg for VaR inputs only the Delta plugin streams vectors	NOT BUILT
Official publish to S3 local or EFS Delta root only; s3:// fails	NOT BUILT



To reach 1.3M positions and 100,000 portfolios, still to build

1 The 1M × 50 benchmark (and 1.3M) on a real server
2 VaR sharding across Drishti*net (scatter-gather)
3 The lake layout at scale: sort, ZSTD, file sizes
4 Portfolio vectors in a batch; > 100,000 rows read
5 Iceberg for VaR inputs (vectors from the Iceberg plugin)
6 Official results published to s3:// (local/EFS today)
7 Several consoles without sticky sessions

Numbers of record you can reproduce, and what comes next

Official, append-only

Every MGMT and LEGAL node, every date, from the morning run; a restatement adds a result version. Published to risk-results/var-official (Delta, appendOnly) with JSONL beside it.

Reproducible

Each answer names its inputs' data versions, batch ids, scenario set, position rules' digest, membership hash and engine version; a capture keeps the vector itself.

Kept 7 years

Vectors and their time travel, official captures and records; lake maintenance never vacuums the risk domains below 7 years.

Entitled and audited

Per role and node: positions or aggregate; masks on rows; every headline, capture, basket, reload and official run audited.

New in 2.9.4, built

- Any input contract: PVs or P&L, by trade or position
- Any reporting currency; five built ahead
- Intraday from files and Kafka micro-batches
- Window, confidence and root name as settings
- JSONL everywhere; the PV scripts (tools/pvgen)

Planned in the VaR guide

- Sharding across Drishti*net; the 1M × 50 run on a server
- The lake layout at bank scale
- What-if pricing through QLRisk, once its API is stable
- Stressed set SVAR-250; age-weighted and filtered methods; 10-day VaR; ES backtests
- Cubes, overnight precomputation; the common risk lake

Key insight Official numbers come only from full runs, and each can be recomputed from its inputs years later.

Thank you

Any data. Any domain. One grammar.

One engine, every input: PVs or P&L, by trade or by position, any currency, files or Kafka.

Drishti • Ashutosh Sinha • October 2026 • VaR engine 2.9.4

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.